

Reservoir Computing

Lili Pan
Ronan Brady



AM3064/AM6015 Written Report

School of Mathematical Sciences
University College Cork
Ireland
March 2026

This report is wholly the work of the authors, except where explicitly stated otherwise. The source of any material which was not created by the authors has been clearly cited. The regulations of University College Cork concerning plagiarism have been read and understood.

Date: March 27, 2026

Signature: Lili Pan, Ronan Brady

Contents

1	Introduction	5
2	Motivation	6
2.1	Neural Networks	6
2.2	Recurrent Neural Networks	7
2.2.1	Efficiency of RNNs	7
2.3	Reservoir Computing	8
2.3.1	Structure of RC	8
3	Implementation and Formulation	9
3.1	Training the RC	9
3.1.1	Listening Stage	10
3.1.2	Training Stage	10
3.2	Closed-loop Prediction	11
3.3	Discrete Time Formulation	12
3.4	Summary	12
4	Historical Development and Specific Models of Reservoir Computing	14
4.1	Historical Development of Reservoir Computing	14
4.2	Echo State Networks and Liquid State Machines	14
5	Implementation and Analysis of RC	16
5.1	Listening stage	16
5.2	Training stage	16
5.3	Predicting stage	17
5.4	Analysis	17
5.5	Parameter optimisation	19
6	Applications	21
6.1	Chaotic Time-Series Prediction	21
6.2	Lorenz63 System	22
7	Conclusions	26
A	Numerical Methods	29
A.1	Runge-Kutta 4th Order	29
B	Lorenz System ESN Example	30

B.1	Import required packages	30
B.2	Computing the Lorenz System via numerical methods	30
B.3	Dependent functions for ESN implementation	31
B.4	Tune parameters	33
B.5	Apply ESN	33
B.6	Prepare data for comparisons in plots	34
B.7	Plot input signal $u(t)$	34
B.8	Plot component-wise true data vs predicted data	35
B.9	A 3D plot of true data vs predicted data	36
B.10	Computing the Root Mean Square Error of true data vs predicted data .	37
B.11	Plot of standardised residuals until cutoff	38
C	NARMA10 ESN Example & Parameter Optimisation	40
C.1	ESN class	40
C.2	NARMA10 example	42
C.3	RC Implementation	43
C.4	Closed-Loop Prediction and Parameter Optimisation	46

Author contributions

Author #1 [Lili Pan] wrote chapters 4, 5, 6.1, 7 and the code in appendix C.

Author #2 [Ronan Brady] wrote chapters 1, 2, 3, 6.2 and appendix A, B.

Chapter 1

Introduction

In this report, the theory, practicalities and applications of Reservoir Computing, a powerful framework for modelling temporal dependencies within the field of machine learning is examined.

In the next chapter, the key motivation behind the development of Reservoir Computing is introduced, beginning with an overview of both recurrent and non-recurrent neural networks and how Reservoir Computing was developed as a means to address the limitations of traditional learning problems.

Chapter 3 provides insight into the mathematical formulation and theory behind reservoir computing by detailing the listening, training and closed loop prediction stages of the process. The chapter also introduces the discrete-time formulation that is used in computation.

Chapter 4 focuses on two specific models of Reservoir Computing; the Echo State Network and Liquid State Machines and how the unification of the two models under the term Reservoir Computing led to the expansion of the field.

The implementation process is again reviewed in chapter 5, but with a focus on the optimisation of parameters and an analysis of the performance of Reservoir Computing.

Finally, in chapter 6, practical applications of the framework are reviewed with an emphasis on chaotic time series prediction. Specific experiments on NARMA10 and the Lorenz System are conducted in this section to illustrate the effectiveness in modelling complex dynamics.

Chapter 2

Motivation

In the field of machine learning, methods for solving non-temporal problems have been extensively studied and optimized, such as feed-forward neural networks. Non-temporal problems are problems in which individual observations used in training the network are assumed to be independent of each other. However, in many real world problems, these assumptions may not hold, and so these existing methods are not adequate solutions to these problems. Temporal systems such as climates, financial data and other time series rely on the assumption that each state of the system depends on its previous states. This dependence makes temporal systems unsuitable for models that assume independent observations.

2.1 Neural Networks

Artificial neural networks are machine learning models that mimic the operations of the human brain. A model consists of interconnected "neurons", often referred to as nodes that have the ability to process and learn complex patterns. Nodes are connected via a weight that determines the strength of the connection between nodes. The most fundamental model of neural network is the feed-forward neural network. That maps an input layer through one or more hidden layers to an output. In these models, inputs are assumed to be independent of each other, and more geared towards learning non-temporal problems.

The potential behind neural networks rely on the networks activation function's ability to act as a universal approximator [3]. This has been proved extensively over the years, first in 1989 [1] for sigmoid functions, and notably, for non-polynomial activation functions in 1991 [2]. Universal approximations rely on a continuous, bounded and non-constant activation function that can approximate any continuous function on a compact subset to any desired degree of accuracy if and only if the activation function is non-polynomial.

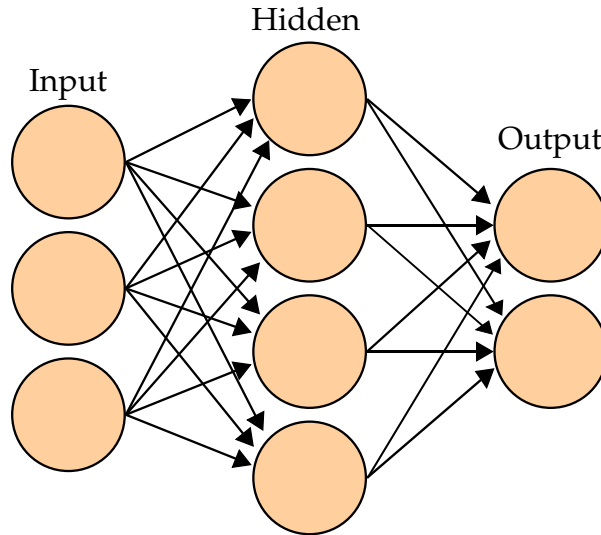


Figure 2.1: An example of a feed-forward neural network with a single hidden layer.

2.2 Recurrent Neural Networks

Recurrent neural networks (RNNs) were developed as a means to address this problem and are a natural solution for modelling temporal dependencies.

RNNs work by maintaining a hidden state that is a function of its previous state and its current input. Essentially, each state relies on information from its previous states and its current input. This property serves as a convenient way of modelling temporal dependencies. RNNs are desirable as they can better approximate non-linear dynamical systems. This may better reflect the data that is being modelled.

Compared to non-recurrent neural networks, the training algorithm behind RNNs use back-propagation through time (BPTT), where after each output is computed, recurrent weights are re-adjusted according to their error. In theory, this algorithm is very well suited to the problem, however there are a few pitfalls in practice.

2.2.1 Efficiency of RNNs

Due to the fact that this computation takes place after each output is computed, when working with large training data sets, optimization can become very computationally expensive very quickly.

A well known problem with RNNs is that they are often subject to training instability which arises from repeated iterations of gradient descent algorithm when back propagating the gradients. This can lead to exploding or vanishing gradients which prevents the network from learning long sequential dependencies. [16]

In summary, training fully recurrent connections can be unstable and inefficient, so we ask the following question; is it necessary to train *all* recurrent connections? Is there a way to exploit the rich dynamics of RNNs without the computational cost of training every internal weight? These questions led to the formation of reservoir computing.

2.3 Reservoir Computing

Reservoir Computing (RC) offers an efficient learning framework for recurrent neural networks. A reservoir computing system typically consists of an input that is mapped into a higher-dimensional, fixed, non-linear reservoir with sparse connections. In comparison to traditional machine learning methods, only the output, also known as the readout layer, is trained. As a result, the computational expense of training is significantly reduced. The learning algorithm reduces to a simple method, such as ridge regression. In short, RC simplifies recurrent learning by shifting the complexity from optimizing to dynamic representation.

2.3.1 Structure of RC

Reservoir Computing follows a similar structure that of a traditional recurrent neural networks. An input is mapped into a reservoir which is then mapped into a readout or output layer.

The input represents a temporal signal that contains sequential dependencies. This could be something such as time series, speech, sequential decision making, etc.

The reservoir is a random recurrent dynamical system. Inputs are projected into a higher-dimensional non-linear state space that transforms temporal signals into rich dynamics.

Both the input and reservoir connections are randomly initialised and fixed throughout the training process. The idea behind randomizing the reservoir connections on initialization is to be able to represent the input in many different non-linear combinations.

The state of the reservoir is updated through time and represents both the current state and previous states of the reservoir, which enables the reservoir to have a memory property that fades over time as more inputs and reservoir states are embedded. Hence, the reservoir implicitly encodes the input signal where more recent inputs have a stronger influence and the influence of older inputs gradually fade over time. [12]

Chapter 3

Implementation and Formulation

Define $u(t) \in \mathbb{R}^K$ as the input, $r(t) \in \mathbb{R}^N$ as the state of the reservoir, and $y(t) \in \mathbb{R}^L$ the output, all at time t . Here, K and L represent the input and output dimensions respectively. N denotes the number of internal reservoir nodes. We will assume we have access to a particular input signal at time t for $0 < t \leq T$.

We will also consider the following weight matrices:

1. $W^{in} \in \mathbb{R}^{N \times K}$: connections between input and internal units of the reservoir.
2. $W^{res} \in \mathbb{R}^{N \times N}$: connections between internal units.
3. $W^{out} \in \mathbb{R}^{L \times (N+K+1)}$: connections between reservoir units to output units.

As mentioned previously, W^{in} and W^{res} have randomly initialised weights that are sparsely connected. These matrices are fixed during the process. The goal is to train a readout matrix W^{out} that should be able to represent the complex patterns of any dynamical system.

3.1 Training the RC

The training of the RC consists of two stages; the listening and training stages. In both stages, we define a reservoir that is updated forward in time using $u(t)$.

We will define the state of the reservoir as

$$\dot{r}(t) = \gamma[-r(t) + f(W^{res}r(t) + \sigma W^{in}u(t))] \quad (3.1)$$

$$r(0) = r_0 \quad (3.2)$$

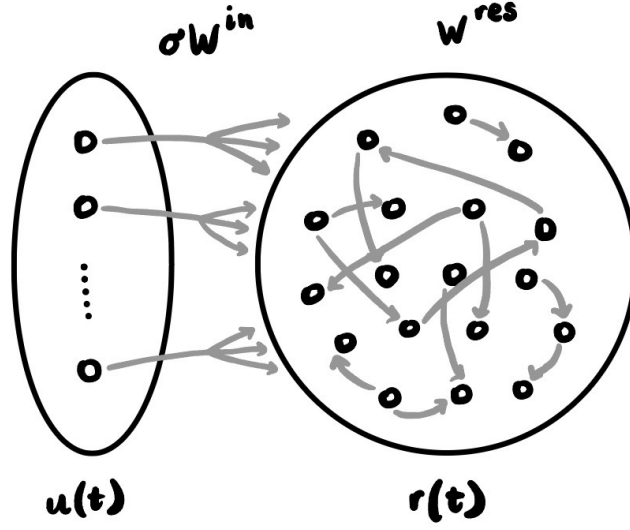


Figure 3.1: The input $u(t)$ is projected by σW^{in} when updating $r(t)$. [5]

3.1.1 Listening Stage

As the reservoir is a recurrent system, its initial state may influence early output and capture dynamics that do not reflect the longer term dynamics of the system. We aim to address this by employing a listening period, also referred to as a washout period.

The reservoir is driven away from its initial state r_0 , which is typically chosen to be the zero vector. During this period, the corresponding outputs are discarded. Training only begins after the initial transient effects fade. We will denote the arbitrary time for these effects to fade as $t_{listen} > 0$.

Hence, $r(t)$ is updated for $0 < t \leq t_{listen}$, according to 3.1 [5].

3.1.2 Training Stage

After the listening stage has concluded, dependence on initial conditions has vanished, and now we collect output data used for training and store in a vector $x(t)$ where at each step, $x(t)$ is defined as

$$x(t) = \begin{bmatrix} 1 \\ u(t) \\ r(t) \end{bmatrix} \quad (3.3)$$

Again, $r(t)$ is updated according to 3.1 for $t_{listen} \leq t \leq t_{train} = T$. Now, as we are only capturing the long term dynamics, we begin to store reservoir states in a vector. The idea is to minimise the mean squared error,

$$mse_{train} = \int_{t_{listen}}^{t_{train}} \frac{1}{t_{train} - t_{listen}} |u(t) - \hat{u}(t)|^2 dt, \quad (3.4)$$

where $\hat{u}(t)$ denotes the predicted output of the system at time t .

Training of W^{out} reduces to a simple mechanism such as ridge regression which will be used in examples later in this report. The use of ridge regression as the training method has the advantage of avoiding overfitting as the ridge regularisation parameter, β_{ridge} , acts as a penalty to weights with a relatively high magnitude.

$$W^{out} = YX^T(XX^T + \beta_{ridge}I)^{-1} \quad (3.5)$$

X acts as the design matrix of form $X = [x(t_{listen}), \dots, x(t_{train})]$, the concatenation of $x(t)$ vectors. Y is the target data, and I is an identity matrix of size $N \times N$. [5]

3.2 Closed-loop Prediction

The goal is to use the now trained W^{out} to generate an autonomous dynamical system. In other words, we wish to be able to perform closed-loop prediction where we are no longer relying on external input and instead use our predicted output to evolve the system autonomously.

The closed-loop prediction evolves for $t > t_{train} = T$ according to

$$\dot{\hat{r}}(t) = \gamma[-\hat{r}(t) + f(W^{res}\hat{r}(t) + \sigma W^{in}\hat{u}(t))] \quad (3.6)$$

$$\hat{r}(t_{train}) = r(t_{train}) \quad (3.7)$$

$\hat{r}(t)$ represents the predicted state of the reservoir using the predicted input signal $\hat{u}(t)$ at time t .

After the the reservoir is trained, we should have been able to get a reconstruction of the target attractor of the dynamical system.

Our predicted output can then be computed with

$$\hat{u}(t) = W^{out}\hat{r}(t) \quad (3.8)$$

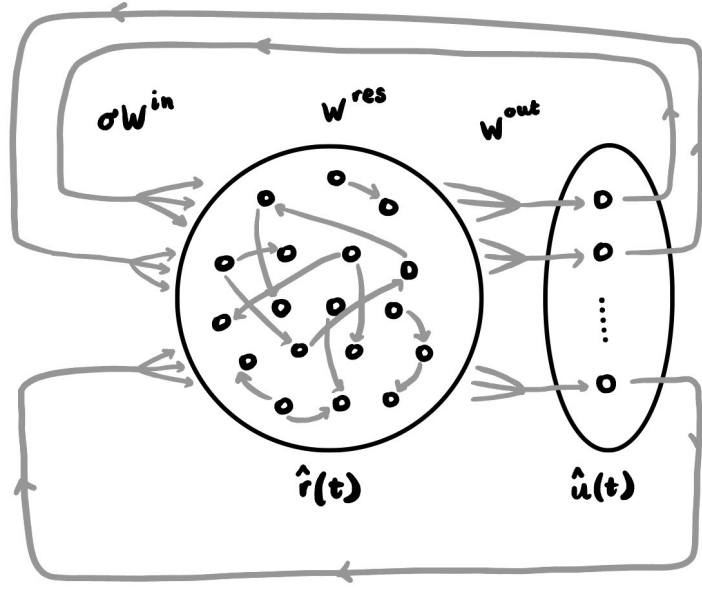


Figure 3.2: Closed loop prediction of $\hat{u}(t)$, where $\hat{r}(t)$ is updated according to 3.6.

3.3 Discrete Time Formulation

When using real world data as $u(t)$, it is worth noting that one may only have access to data at discrete time steps. For example, a weather station may have measured the temperature at a particular point at ten minute time intervals over a specific time period. A discrete time implementation of reservoir computing is very similar to its continuous counterpart.

A general formulation of a discrete time reservoir computer is as follows:

$$r(t+1) = f(W^{res}r(t) + \sigma W^{in}u(t)) \quad (3.9)$$

and

$$\hat{u}(t+1) = W^{out}\hat{r}(t) \quad (3.10)$$

after W^{out} has been trained. Of course in discrete time, time moves in discrete steps so denote time t as a non-negative integer, $t \in \mathbb{Z}_{\geq 0}$.

3.4 Summary

Reservoir computing consists of three essential components; a non-linear recurrent dynamical system, high-dimensional state representation of a temporal input signal, and a simple, typically linear, readout layer that reduces computation expense and the need to train every recurrent connection. Reservoir computing shifts the difficulty from optimizing recurrent weights to designing expressive reservoir dynamics. From

the mathematical formulation, we have identified two fundamental ideas; stability of recurrent dynamics (echo state property), and computational richness of transient states (separation and approximation). In the following sections, we will analyse how these ideas emerged.

Chapter 4

Historical Development and Specific Models of Reservoir Computing

4.1 Historical Development of Reservoir Computing

Reservoir Computing developed from two independent research directions: Echo State Networks proposed by Jaeger [6] and Liquid State Machines proposed by Maass et al. [7]. Although they were introduced in different contexts, both models were based on the same key idea: using a fixed recurrent dynamical system together with a trainable readout layer [6, 7]. This was an important shift in recurrent learning, because it avoided the need to train all recurrent connections directly and instead relied on the reservoir to generate a rich state representation of the input.

In 2007, Schrauwen et al. [8] unified these related ideas under the term Reservoir Computing, which established RC as a broader computational framework rather than two separate model classes. It moved the focus away from individual architectures and towards a more general principle: a fixed nonlinear dynamical system can be used as a computational resource if its transient responses are sufficiently informative.

Later work further extended Reservoir Computing to physical and hardware-based systems, including optoelectronic reservoirs and soft-body reservoirs [9, 10, 11], illustrating that RC is not limited to standard software neural networks, but can be understood more broadly as computation with dynamical systems.

4.2 Echo State Networks and Liquid State Machines

As the two main early models of Reservoir Computing, Echo State Networks and Liquid State Machines emphasise different aspects of the RC idea. ESNs are usually formulated in discrete time and use continuous-valued reservoir states, while LSMs are typically formulated in continuous time and are based on spiking neurons [6, 7].

ESNs are more commonly used in machine learning and engineering oriented applications. Their central theoretical idea is the *Echo State Property*, which requires that the influence of the initial reservoir state should gradually disappear so that the reservoir state becomes mainly determined by the input history [6]. By contrast, LSMs

arose from computational neuroscience and place greater emphasis on computation through transient liquid responses and high-dimensional dynamic trajectories. Their main theoretical requirements are the *Separation Property* and the *Approximation Property*, which require that different inputs generate distinguishable internal states and that the readout can approximate the desired output [7].

In ESNs, the spectral radius of the reservoir matrix is an especially important design parameter because it controls the strength of internal feedback. If the spectral radius is too small, the reservoir may lose memory too quickly and fail to capture temporal dependencies. If it is too large, the dynamics may become unstable, making closed-loop prediction less reliable. Therefore, the spectral radius is a useful practical heuristic for balancing memory and stability in ESN design, and this is also one of the key parameters examined later in Chapter 4.

Table 4.1 summarises the main similarities and differences between ESNs and LSMs.

Table 4.1: Comparison between Echo State Networks and Liquid State Machines

Aspect	ESN	LSM
Origin	Machine learning	Computational neuroscience
State model	Discrete-time recurrent update	Continuous-time liquid dynamics
Neuron type	Usually continuous valued units	Usually spiking neurons
Typical formulation	Explicit matrix recurrence	Filter-based or liquid-state formulation
Memory mechanism	Echoes of past inputs in reservoir state	Transient liquid response with fading memory
Key condition	Echo State Property	Separation and Approximation Properties
Training	Only the readout is trained	Only the readout is trained
Typical interpretation	Engineering-friendly reservoir model	Biologically inspired real-time computing model

As shown in Table 4.1, ESNs and LSMs share the same basic Reservoir Computing philosophy, but differ in formulation, interpretation, and emphasis.

The later implementation of RC in this report is based on the ESN framework, since it is more suitable for analysing spectral radius, washout, and closed-loop prediction in a simple and practical setting.

Chapter 5

Implementation and Analysis of RC

This chapter explains how the mathematical formulation introduced in Chapter 2 is translated into a practical Reservoir Computing experiment. The implementation is organised into three stages: listening, training, and forming closed-loop and predicting. Consistent with the standard RC framework, only the readout layer is trained, while the input and reservoir weights remain fixed after random initialisation.

In this experiment, both the input dimension and output dimension are set to 1, and the reservoir size is set to $N = 200$. The input weight matrix W_{in} is randomly generated and scaled by $\sigma_{in} = 0.4$, while the reservoir matrix W_{res} is randomly generated with sparsity 0.1 and then rescaled to a target spectral radius of 0.7. The ridge regularisation coefficient is set to $\beta_{ridge} = 10^{-4}$. These parameters determine the internal dynamics of the reservoir and influence the balance between memory, stability, and prediction performance.

5.1 Listening stage

In the listening stage, the reservoir is driven by the input signal for an initial washout period in order to reduce the influence of the zero initial state. Starting from $r(0) = 0$, the reservoir state is updated step by step according to the discrete-time equation introduced earlier. In this experiment, the washout length is set to $t_{listen} = 100$, so the first 100 time steps are used only to evolve the reservoir state and are not included in training. The purpose of this stage is to allow the internal reservoir dynamics to become more representative of the input history before the readout is fitted.

5.2 Training stage

After the listening stage, the experiment enters the training stage. From step 100 to step 1200, the reservoir state continues to be updated using the input sequence, and the corresponding states are collected for readout training. At each time step, a design vector is formed as

$$x(t) = \begin{bmatrix} 1 \\ u(t) \\ r(t) \end{bmatrix},$$

which combines a bias term, the current input, and the reservoir state. These design vectors are stacked to construct the design matrix

$$X = [x(t_{listen}), x(t_{listen} + 1), \dots, x(t_{train})].$$

The target outputs are collected in the corresponding target matrix Y . The readout matrix W_{out} is then computed using ridge regression,

$$W_{out} = YX^T(XX^T + \beta_{ridge}I)^{-1}.$$

This stage is computationally efficient because only the readout weights are optimised, while the recurrent reservoir remains fixed.

5.3 Predicting stage

After training, the learned readout is used in a recursive closed-loop prediction stage. In this experiment, only the first input value is taken from the true signal, while all later inputs are replaced by the model's own predicted outputs. More specifically, after the final training state is obtained, the model generates

$$\hat{y}(t) = W_{out}x(t),$$

and then feeds this predicted value back as the next input. In this way, the system evolves autonomously for 400 prediction steps. This stage provides a more demanding test of the RC model, because prediction errors are no longer corrected by the true signal and may accumulate over time.

5.4 Analysis

The experiment is performed on a one-dimensional signal defined as

$$s(t) = \sin(t) + 0.5 \sin(2.3t) + 0.2 \cos(0.7t),$$

with a total length of 2000 time steps.

Under the initial parameter setting, the model matches the true signal reasonably well at the beginning of the prediction interval. However, as the recursive prediction continues, the generated trajectory gradually departs from the true signal and the error increases over time. This suggests that the current implementation is effective for short-term prediction, but less reliable over longer horizons.

This behaviour is affected by the reservoir parameters. The spectral radius influences the recurrent feedback, the input scaling controls how strongly the signal drives the reservoir states, and the ridge coefficient affects the stability of the readout. The reservoir size and washout length also influence the quality of the state representation. Therefore, parameter tuning is needed to improve recursive prediction performance.

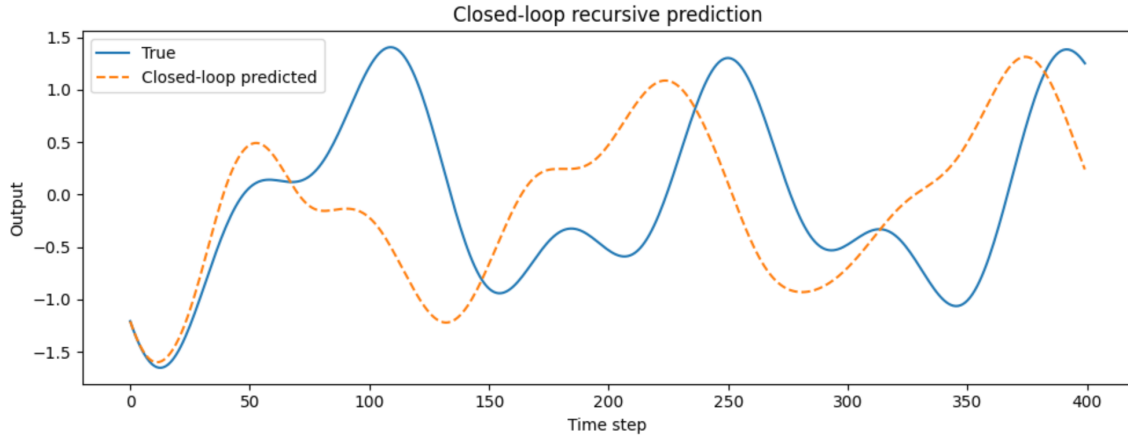


Figure 5.1: Closed-loop recursive prediction under the initial parameter setting.

Figure 5.1 compares the true signal with the closed-loop recursive prediction under the initial parameter setting. The predicted curve follows the true signal reasonably well at the beginning of the forecasting interval, but gradually deviates as the prediction continues. Visible differences in amplitude, phase, and peak locations appear over time, showing that the model performs better in the short term than over longer recursive horizons.

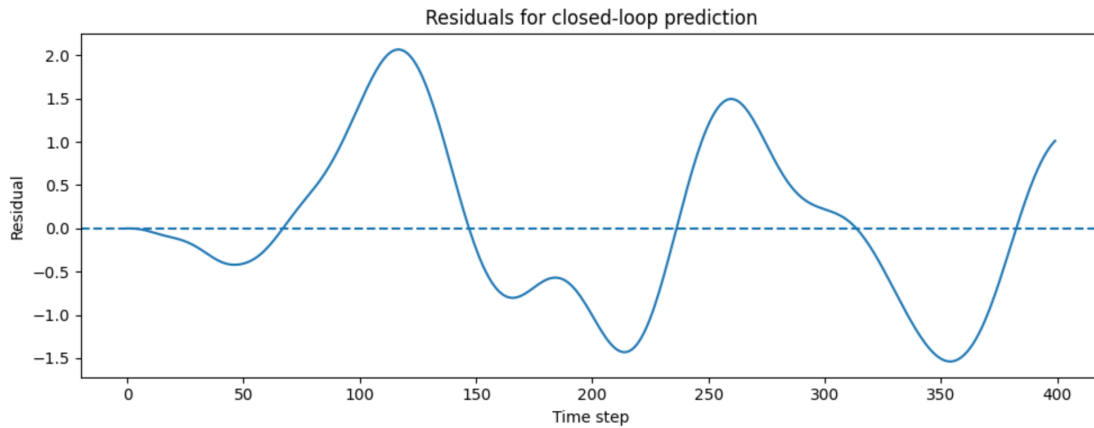


Figure 5.2: Residuals of the closed-loop recursive prediction under the initial parameter setting.

Figure 5.2 shows the residuals of the closed-loop recursive prediction. The residuals are relatively small at the beginning of the forecasting interval, but become larger and more structured over time. This indicates that the prediction error is systematic rather than purely random, with both amplitude error and phase shift appearing in longer-horizon prediction.

5.5 Parameter optimisation

To improve recursive prediction performance, a grid search was performed over the main RC parameters. The reservoir size was tested over $N \in \{100, 200, 300\}$, the input scaling over $\sigma_{in} \in \{0.2, 0.3, 0.4\}$, the spectral radius over $\rho(W_{res}) \in \{0.5, 0.6, 0.7, 0.8\}$, the sparsity over $\{0.05, 0.1\}$, the ridge coefficient over $\beta_{ridge} \in \{10^{-4}, 10^{-3}, 10^{-2}\}$, and the washout length over $t_{listen} \in \{50, 100, 150\}$. This resulted in a total of 648 parameter combinations.

For each parameter combination, the complete three-stage RC experiment was repeated and the resulting RMSE, NMSE, and R^2 values were recorded. The final parameter setting was selected primarily based on the lowest RMSE, while NMSE and R^2 were also checked to confirm that the selected configuration gave a consistent improvement. RMSE was used as the primary selection criterion because the signal scale is fixed and directly interpretable.

Table 5.1 shows a clear improvement after parameter optimisation. The RMSE decreases from 1.0207 to 0.4787, which is a reduction of about 53.1%, while the NMSE decreases from 1.5629 to 0.3437, corresponding to a reduction of about 78.0%. At the same time, R^2 increases from -0.5629 to 0.6563. This is particularly important because the negative R^2 for the initial setting indicates that the recursive prediction performed worse than a simple mean-based baseline, whereas the optimised setting explains about 65.6% of the variance in the target signal. Overall, these results suggest that parameter tuning substantially improves the quality of recursive prediction in this experimental setting.

Table 5.1: Comparison between the initial parameter setting and the optimised parameter setting.

Setting	RMSE	NMSE	R^2
Initial setting	1.0207	1.5629	-0.5629
Optimised setting	0.4787	0.3437	0.6563

The best result was obtained with $N = 300$, $\sigma_{in} = 0.2$, $\rho(W_{res}) = 0.8$, sparsity = 0.1, $\beta_{ridge} = 10^{-3}$, and $t_{listen} = 50$. Under this parameter setting, the model achieved RMSE = 0.4787, NMSE = 0.3437, and $R^2 = 0.6563$.

The top-ranked parameter settings in Table 5.2 suggest some general patterns. Larger reservoir sizes appear frequently among the stronger results, especially $N = 300$. Lower input scaling values, particularly $\sigma_{in} = 0.2$, also appear repeatedly. In addition, spectral radius values around 0.8 occur often in the best performing settings. These observations provide useful guidance for this implementation.

Table 5.2: Top 10 parameter settings ranked by RMSE in the grid search.

Rank	N	σ_{in}	$\rho(W_{res})$	Sparsity	β_{ridge}	t_{listen}	RMSE	R^2
1	300	0.2	0.8	0.10	0.0010	50	0.4787	0.6563
2	300	0.2	0.7	0.10	0.0001	50	0.7214	0.2193
3	200	0.2	0.8	0.05	0.0001	150	0.7377	0.1837
4	300	0.2	0.8	0.10	0.0001	150	0.7452	0.1670
5	100	0.4	0.8	0.10	0.0010	50	0.7457	0.1658
6	300	0.3	0.8	0.10	0.0010	50	0.7626	0.1275
7	200	0.2	0.8	0.05	0.0001	100	0.7679	0.1154
8	100	0.2	0.5	0.05	0.0100	150	0.7789	0.0900
9	100	0.2	0.8	0.10	0.0001	50	0.7876	0.0694
10	100	0.4	0.8	0.10	0.0010	100	0.7943	0.0535

Figure 5.3 shows the recursive prediction under the optimised parameter setting. Compared with the initial result, the predicted trajectory remains closer to the true signal over the forecasting horizon, indicating improved prediction accuracy.

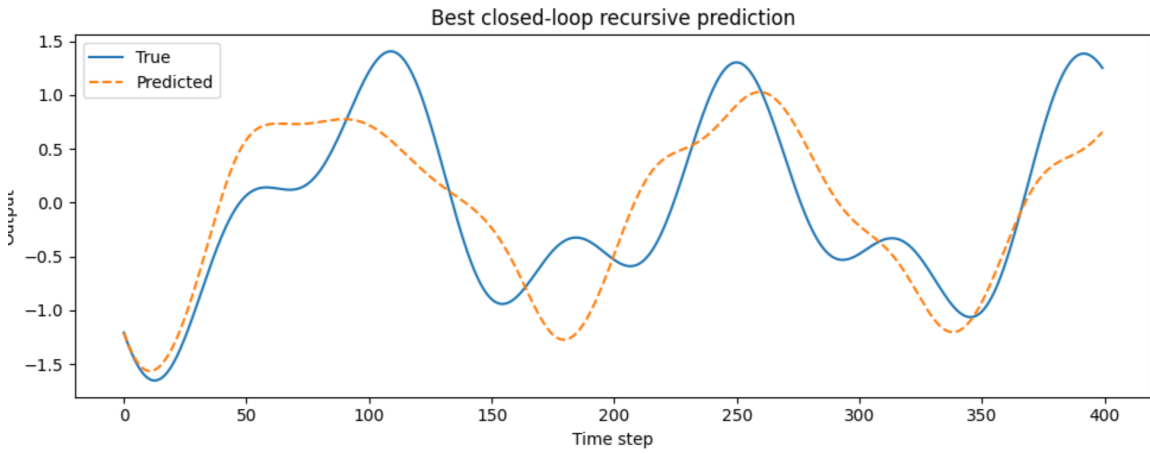


Figure 5.3: Closed-loop recursive prediction under the optimised parameter setting.

Chapter 6

Applications

The Reservoir Computing approach to machine learning has been successfully applied to a wide range of problems, including time-series prediction, visual identification tasks, real-time detection of epileptic seizures, inference of unmeasured state variables from limited time-series data, estimation of Lyapunov exponents, and analysis of causal dependencies between variables[5]. In this chapter, the focus is placed on nonlinear and chaotic temporal modelling, which is particularly relevant to the aims of this report. The following two examples illustrate how Echo State Networks can capture important temporal dependencies and dynamical structure in practice.

6.1 Chaotic Time-Series Prediction

As mentioned above, one of the main problems that RC solves is being able to learn from limited time series data.

Some examples include weather and climate related forecasting, chaotic benchmark systems like Lorenz type dynamics and energy demand and financial data. Reservoir Computing works well with these tasks because the reservoir dynamics naturally model non-linear systems. The high-dimensional state expansion captures complex patterns and the training remains computationally simple.

In experiments reported by Jaeger and Haas in 2004, Echo State Networks were applied to chaotic time series prediction.

Reservoir Computing uses non-linearity to learn complex temporal patterns efficiently, and the reported prediction accuracy improved by approximately 2,400 times compared to previous techniques. [13]

Fig 6.1 provides a look at how ESNs perform using NARMA10 as a benchmark. The prediction follows the overall trend, showing that ESNs can indeed capture long-term dependencies. However, some peaks are missed or underestimated, so the result is not perfect. The normalised mean squared error of this result is 0.346.

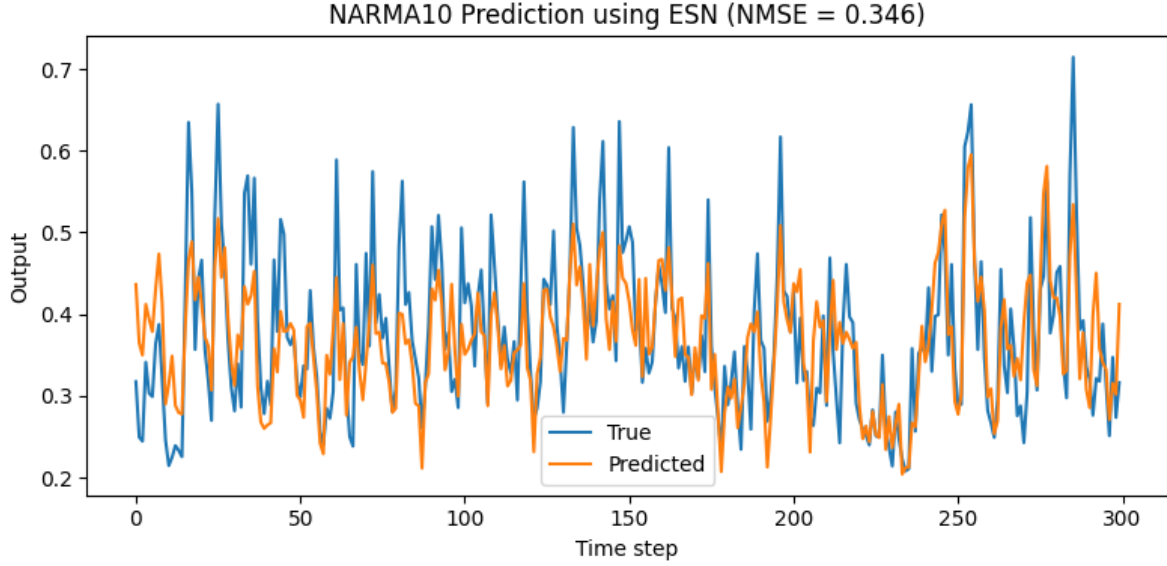


Figure 6.1: Echo State Network approach to Reservoir Computing applied to NARMA10 to predict 300 time steps.

6.2 Lorenz63 System

In this example, we will look at how an ESN can be used to capture the chaotic dynamics of the famous Lorenz System. We will consider the three coupled ordinary differential equations developed by Edward Lorenz back in 1963 [14] to model atmospheric convection

$$\dot{u}_1 = \sigma(u_2 - u_1) \quad (6.1)$$

$$\dot{u}_2 = u_1(\rho - u_3) - u_2 \quad (6.2)$$

$$\dot{u}_3 = u_1 u_2 - \beta u_3 \quad (6.3)$$

This system has a strange attractor for certain parameter values, and we wish to evaluate how an ESN can learn and reproduce its long term dynamics. The parameters and initial conditions that will be used in the Lorenz System for this example are shown in table 6.1.

$u_1(0)$	$u_2(0)$	$u_3(0)$	σ	ρ	β
0.1	0.1	0.1	10	28	$\frac{8}{3}$

Table 6.1: The parameters used in the Lorenz System for this example.

Consider an ESN with a three-dimensional input and output signal, where our input signal is shown in figure 6.2. The reservoir itself is of course randomly initialised, and additionally rescaled to a target spectral radius of 0.9. Code for the implementation can be found in B.

Figure 6.2 displays a component-wise plot of this system for 10,000 time steps, evaluated using the Runge Kutta fourth order numerical method A.1. The initial spike follows from the system settling from the initial conditions that does not reflect the longer term dynamics so a listening period is required to mitigate the effects of these initial conditions. The dotted red line represents the cutoff for this listening stage.

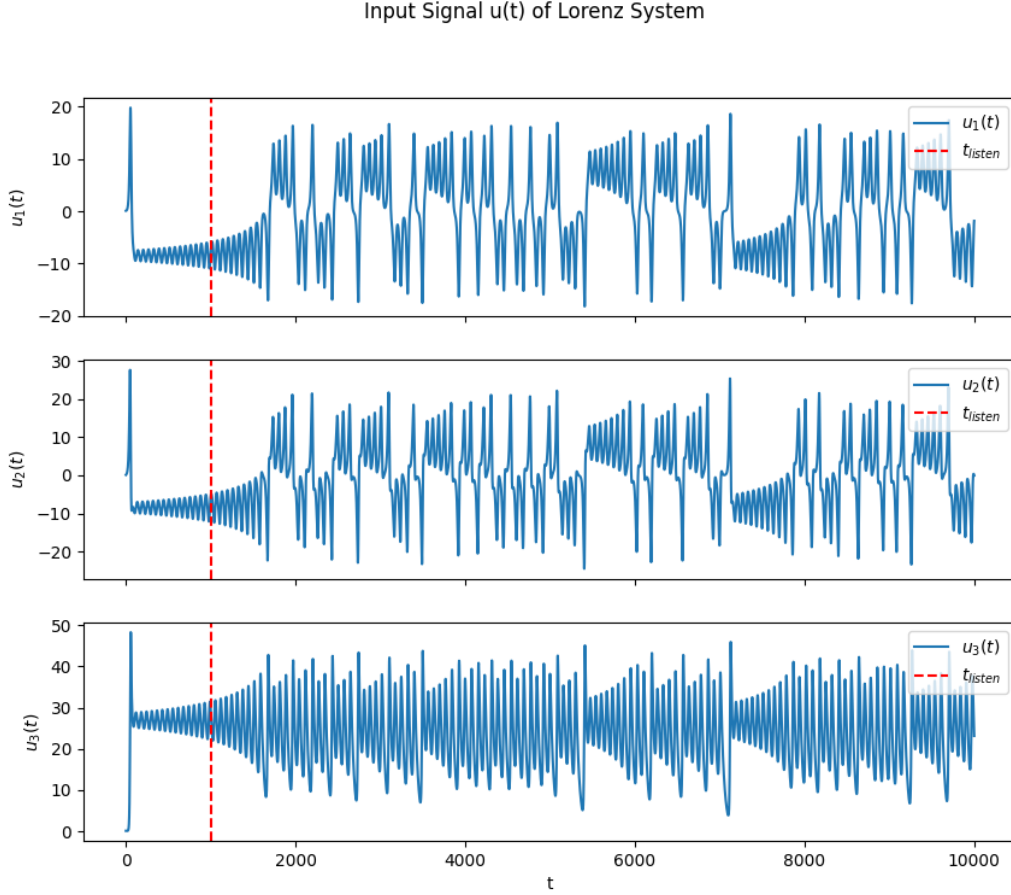


Figure 6.2: The input signal of Lorenz63 system.

Table 6.2 shows the parameters that will be used for the ESN implementation.

N	σ_{in}	γ	β_{ridge}	t_{listen}	t_{train}
500	1	0.6	10^{-3}	1000	10000

Table 6.2: The parameters used in the ESN

The system evolves autonomously for 2,000 time steps and a plot of $\hat{u}(t)$ for $\hat{u}(t)$ for $t_{train} \leq t \leq t_{train} + 2000$ is shown in figure 6.3. The plot shows that the target attractor has been reconstructed. Due to the Lorenz' Systems sensitivity to perturbations, the prediction quickly breaks down but the performance of the ESN can be evaluated until an arbitrary cutoff point chosen to be at $t = 10200$. The computed mean-squared error of this period is shown in table 6.3. These results show us the average error between that of the true values and the predicted values.

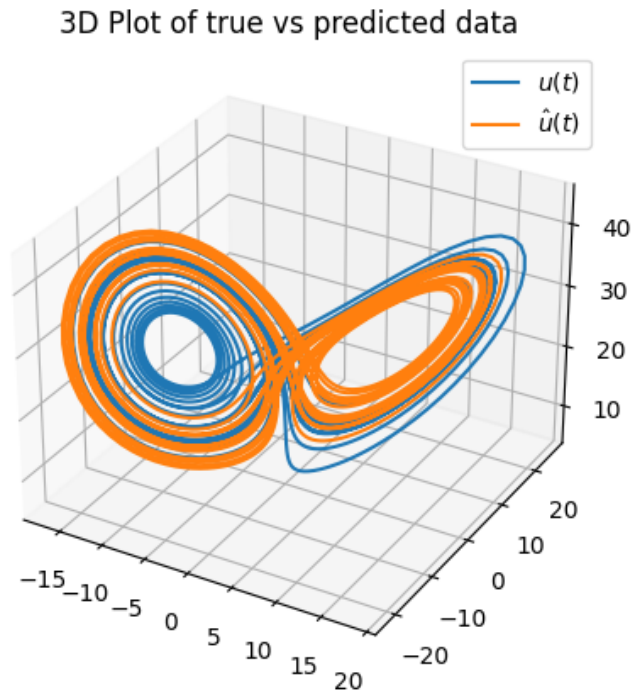
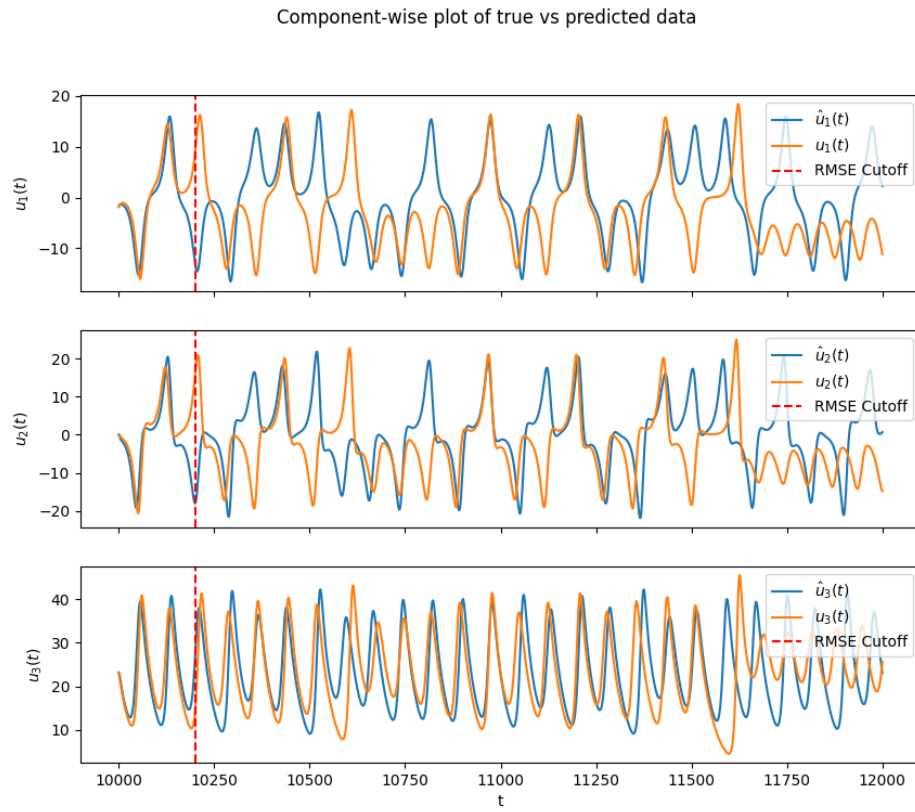


Figure 6.3: 2000 step prediction for the Lorenz System. The red vertical straight line represents the cutoff point for computing the root mean square error at $t = 10200$.

$\text{rmse}(u_1, \hat{u}_1)$	$\text{rmse}(u_2, \hat{u}_2)$	$\text{rmse}(u_3, \hat{u}_3)$
4.57865905	7.54930093	4.90696865

Table 6.3: Root mean squared error of the prediction until RMSE cutoff point.

Fig 6.4 shows a plot of the standardised residuals of the predicted output $\hat{u}(t)$ against the true signal $u(t)$. As seen, the residuals stay relatively close to the mean of 0, and mainly only deviate at the peaks and troughs of the dynamical system.

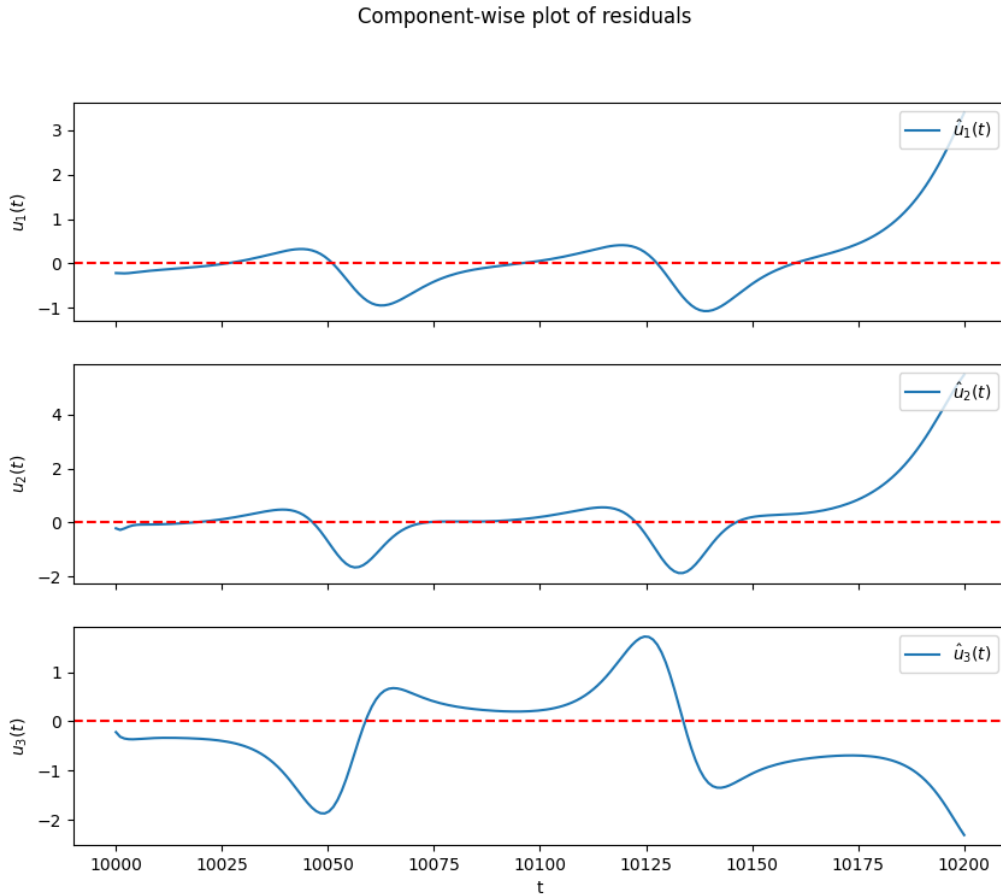


Figure 6.4: Plotted residuals of $\hat{u}(t)$ for $10000 \leq t \leq 10200$

Chapter 7

Conclusions

Reservoir Computing provides an alternative approach to training recurrent systems by avoiding the difficult optimisation of recurrent connections in traditional recurrent neural networks.

Starting from ESNs and LSMs, Reservoir Computing developed into a general framework that connects dynamical systems and machine learning.

In this report, the RC implementation was organised around the three key parts of listening, training, closed-loop and predicting. In the listening stage, the reservoir was driven by the external input so that the influence of the initial state was reduced and the input history was encoded into a high-dimensional state representation. In the training stage, the collected reservoir states were used to learn the readout through ridge regression. In the closed-loop and predicting stage, the trained model was run in a closed loop, using its own predicted output as the next input, so that autonomous prediction could be tested directly. The Chapter 4 experiment showed that the model could follow the target signal reasonably well over short horizons, while parameter optimisation over 648 combinations produced a clear improvement in prediction accuracy. The Chapter 5 examples further suggested that RC can capture important aspects of nonlinear temporal dynamics: in the NARMA10 task, the ESN followed the overall trend but missed some peaks, and in the Lorenz63 example it reproduced the overall structure of the strange attractor over the prediction interval. However, the report also showed that long-term closed-loop prediction remains challenging because errors accumulate over time, and that the results are sensitive to the choice of reservoir parameters.

Bibliography

- [1] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2, 303–314. <https://doi.org/10.1007/BF02551274>
- [2] Leshno, M., Lin, V. Y., Pinkus, A., & Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6), 861–867. [https://doi.org/10.1016/S0893-6080\(05\)80131-5](https://doi.org/10.1016/S0893-6080(05)80131-5)
- [3] Sharma, S., Sharma, S., & Athaiya, A. (2017). Activation functions in neural networks. *Towards Data Science*, 6(12), 310–316.
- [4] Lukoševičius, M. (2012). A practical guide to applying echo state networks. In G. Montavon, G. B. Orr, & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* 2nd ed. pp. 659–686. Springer.
- [5] Flynn, A., Tsachouridis, V. A., & Amann, A. (2021). Multifunctionality in a reservoir computer. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 31(1), Article 013125. <https://doi.org/10.1063/5.0029493>
- [6] Jaeger, H. (2001). *The “echo state” approach to analysing and training recurrent neural networks* (GMD Report No. 148). German National Research Center for Information Technology.
- [7] Maass, W., Natschläger, T., & Markram, H. (2002). Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11), 2531–2560. <https://doi.org/10.1162/089976602760407955>
- [8] Schrauwen, B., Verstraeten, D., & Van Campenhout, J. (2007). An overview of reservoir computing: Theory, applications and implementations. In D. Verleysen Ed. *Proceedings of the European Symposium on Artificial Neural Networks* pp. 471–482.
- [9] Tanaka, G., Yamane, T., Héroux, J. B., Nakane, R., Kanazawa, N., Takeda, S., Numata, H., Nakano, D., & Hirose, A. (2019). Recent advances in physical reservoir computing: A review. *Neural Networks*, 115, 100–123. <https://doi.org/10.1016/j.neunet.2019.03.005>
- [10] Larger, L., Soriano, M. C., Brunner, D., Appeltant, L., Gutierrez, J. M., Pesquera, L., Mirasso, C. R., & Fischer, I. (2012). Photonic information processing beyond Turing: An optoelectronic implementation of reservoir computing. *Optics Express*, 20(3), 3241–3249. <https://doi.org/10.1364/OE.20.003241>
- [11] Nakajima, K., Hauser, H., Kang, R., Gómez, F., & Pfeifer, R. (2015). Infor-

- mation processing via physical soft body. *Scientific Reports*, 5, Article 10487. <https://doi.org/10.1038/srep10487>
- [12] Gallicchio, C. (2018). *Chasing the echo state property*. arXiv. <https://arxiv.org/abs/1811.10892>
- [13] Jaeger, H., & Haas, H. (2004). Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667), 78–80. <https://doi.org/10.1126/science.1091277>
- [14] Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2), 130–141. [https://doi.org/10.1175/1520-0469\(1963\)020<0130:DNF>2.0.CO;2](https://doi.org/10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2)
- [15] Lukoševičius, M., & Jaeger, H. (2009). Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3), 127–149. <https://doi.org/10.1016/j.cosrev.2009.03.005>
- [16] R. Pascanu, T. Mikolov & Y. Bengion, *On the difficulty of training Recurrent Neural Networks*, (2013) 1211.5063, arXiv , <https://arxiv.org/abs/1211.5063>

Appendix A

Numerical Methods

A.1 Runge-Kutta 4th Order

Runge-Kutta 4th Order is a function approximation iterative method that employs Euler's method to approximate a first-order differential equation.

Suppose we are given the initial value problem as follows:

$$\begin{aligned}\frac{dy}{dt} &= f(t, y) \\ y(t_0) &= y_0,\end{aligned}$$

where $y(t)$ is an unknown function of time t . A step size of h is chosen such that $h > 0$. Define for $n \in \mathbb{N}$ the following:

$$\begin{aligned}y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\ t_{n+1} &= t_n + h\end{aligned}$$

where

$$\begin{aligned}k_1 &= f(t_n, y_n) \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + k_1 \frac{h}{2}\right) \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + k_2 \frac{h}{2}\right) \\ k_4 &= f(t_n + h, y_n + hk_3).\end{aligned}$$

Appendix B

Lorenz System ESN Example

B.1 Import required packages

```
[1]: import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits import mplot3d
%matplotlib inline
```

B.2 Computing the Lorenz System via numerical methods

```
[2]: # Lorenz differential equations
def lorenz(x, y, z, sigma, rho, beta):
    dx = sigma * (y - x)
    dy = x * (rho - z) - y
    dz = (x * y) - (beta * z)
    return([dx, dy, dz])

# Runge-Kutta 4th order. Returns the full array of lorenz steps.
def calc_lorenz(steps, init):
    r = init
    rx = r[0][0]; ry = r[0][1]; rz = r[0][2]

    h = 0.01
    t = 0
    for i in range(1, steps):
        t += h

        k1 = lorenz(rx, ry, rz, sigma, rho, beta)
        k2 = lorenz(rx + h*0.5*k1[0], ry + h*0.5*k1[1], rz + h*0.
↪5*k1[2], sigma, rho, beta)
```

```

        k3 = lorentz(rx + h*0.5*k2[0], ry + h*0.5*k2[1], rz + h*0.
↪5*k2[2], sigma, rho, beta)
        k4 = lorentz(rx + h*k3[0], ry + h*k3[1], rz + h*k3[2], sigma, ↪
↪rho, beta)

        rx += h/6*(k1[0] + 2*k2[0] + 2*k3[0] + k4[0])
        ry += h/6*(k1[1] + 2*k2[1] + 2*k3[1] + k4[1])
        rz += h/6*(k1[2] + 2*k2[2] + 2*k3[2] + k4[2])

        r.append([rx, ry, rz])

    return np.array(r)

```

B.3 Dependent functions for ESN implementation

```

[3]: # Standardise a given vector
def standardise(x):
    x = (x - np.mean(x)) / np.std(x)
    return(x)

# Return the spectral radius of a given matrix
def spectral_radius(A):
    eigenvalues = np.linalg.eigvals(A)
    return(max(abs(eigenvalues)))

# Normalises a given matrix
def normalisation(A):
    s = spectral_radius(A)
    if (s == 0):
        print("Error: spectral radius = 0. Unable to normalise.")
        return
    else:
        return(1/s * A)

# Rescale a matrix to a target spectral radius alpha
def rescale(A, alpha):
    return(alpha * A)

# Generates the input and reservoir weight matrices (with a given seed ↪
↪if applicable)
def generate_DR(gen_seed):

    if(gen_seed == 0):
        rng = np.random.default_rng()
    else:
        rng = np.random.default_rng(gen_seed)

```



```

    W_in = rng.uniform(-0.12, 0.12, size = (N,3))
    W = rescale(normalisation(rng.uniform(-1, 1, size = (N,N))),  

    ↪target_radius)

    return(W_in, W)

# Train the W_out matrix in the ESN after a given washout period
def train_esn(U, W, W_in, leak, washout, ridge=1e-3):
    T = U.shape[0]
    N = W.shape[0]

    x = np.zeros(N)
    X = []
    Y = []

    for n in range(T-1):
        x = (1-leak) * x + leak*np.tanh((W @ x) + (W_in @ U[n]))

        if n>= washout:
            z = np.hstack([1, x, U[n]])
            X.append(z)
            Y.append(U[n+1])

    X = np.array(X)
    Y = np.array(Y)

    I = np.eye(X.shape[1])
    I[0, 0] = 0
    W_out = np.linalg.solve((X.T @ X + ridge * I), X.T @ Y)
    return W_out

# Predict the future state of the dynamical system
def predict_esn(u0, W, W_in, W_out, leak, steps):
    N = W.shape[0]
    x = np.zeros(N)
    u = u0.copy()

    preds = []
    preds.append(u)

    for n in range(steps-1):
        x = (1-leak)*x + leak*np.tanh(W @ x + W_in @ u)
        z = np.hstack([1, x, u])
        u = z @ W_out
        preds.append(u)

```

```
return np.array(preds)
```

B.4 Tune parameters

Lorenz parameters

```
[4]: sigma = 10
    rho = 28
    beta = 8/3

    initial_condition = [[0.1,0.1,0.1]] # Initial condition  $u(0)$  for lorenz
    ↪system
```

RC parameters

```
[5]: seed = 4 # Set a seed for reproducibility (set seed to 0 for random)

    N = 500 # Number of internal reservoir units
    leak = 0.6 # Leaky integration parameter
    pred_steps = 2000 # Number of steps to predict
    target_radius = 0.9 # Target reservoir spectral radius to rescale  $W_{res}$ 
    ↪to

    t_listen = 1000 # The washout period
    t_train = 10000 # Number of Lorenz system steps to store

    rmse_cutoff = 200 # Cutoff point when calculating the RMSE of the
    ↪predicted data
```

B.5 Apply ESN

```
[6]: # Generate the dynamic reservoirs for  $W^{in}$  and  $W^{res}$ 
    W_in, W_res = generate_DR(seed)

    # Compute Lorenz System time series data
    input_signal = calc_lorenz(t_train, [[0.1,0.1,0.1]])

    # Train  $W_{out}$  output weights
    W_out = train_esn(input_signal, W_res, W_in, leak, t_listen)

    # Predict the future states
    pred_data = predict_esn(input_signal[-1].copy(), W_res, W_in, W_out,
    ↪leak, pred_steps)
```

B.6 Prepare data for comparisons in plots

```
[7]: # Calculate the true values for the next steps for comparison
initial_condition = [[input_signal[-1][0], input_signal[-1][1],
    ↪input_signal[-1][2]]]
true_data = calc_lorenz(pred_steps, initial_condition)

# Setup the prediction range space
pred_range = np.linspace(t_train, t_train + pred_steps, pred_steps)
```

B.7 Plot input signal $u(t)$

```
[8]: input_range = np.linspace(0, t_train, t_train)

# Plot the input signal
fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

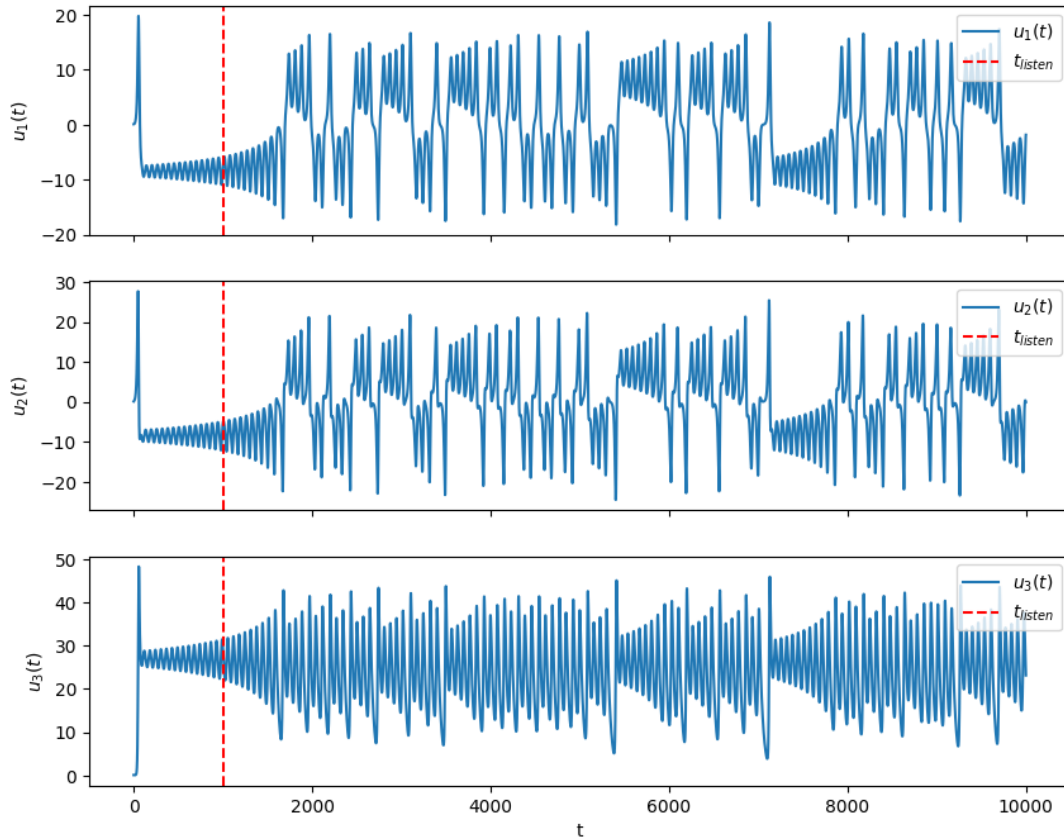
labels = [r'$u_1(t)$', r'$u_2(t)$', r'$u_3(t)$']

fig.suptitle("Input Signal  $u(t)$  of Lorenz System")

for i in range(0,3):
    axes[i].plot(input_range, input_signal[:,i], label=labels[i])
    axes[i].set_ylabel(labels[i])
    axes[i].axvline(x=(t_listen), color='red', linestyle='--',
    ↪label=r'$t_{listen}$')
    axes[i].legend(loc='upper right')

plt.xlabel("t")
plt.savefig("lorenz_input_signal.png")
plt.show()
```

Input Signal $u(t)$ of Lorenz System



B.8 Plot component-wise true data vs predicted data

```
[9]: # Setup the prediction range space
pred_range = np.linspace(t_train, t_train + pred_steps, pred_steps)

# Plot the predicted values against true values
fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

labels_pred = [r'$\hat{u}_1(t)$', r'$\hat{u}_2(t)$', r'$\hat{u}_3(t)$']

fig.suptitle("Component-wise plot of true vs predicted data")

for i in range(0,3):
    axes[i].plot(pred_range, pred_data[:,i], label=labels_pred[i])
    axes[i].plot(pred_range, true_data[:,i], label=labels[i])
    axes[i].axvline(x=(rmse_cutoff + t_train), color='red',
        linestyle='--', label="RMSE Cutoff")
    axes[i].set_ylabel(labels[i])
```

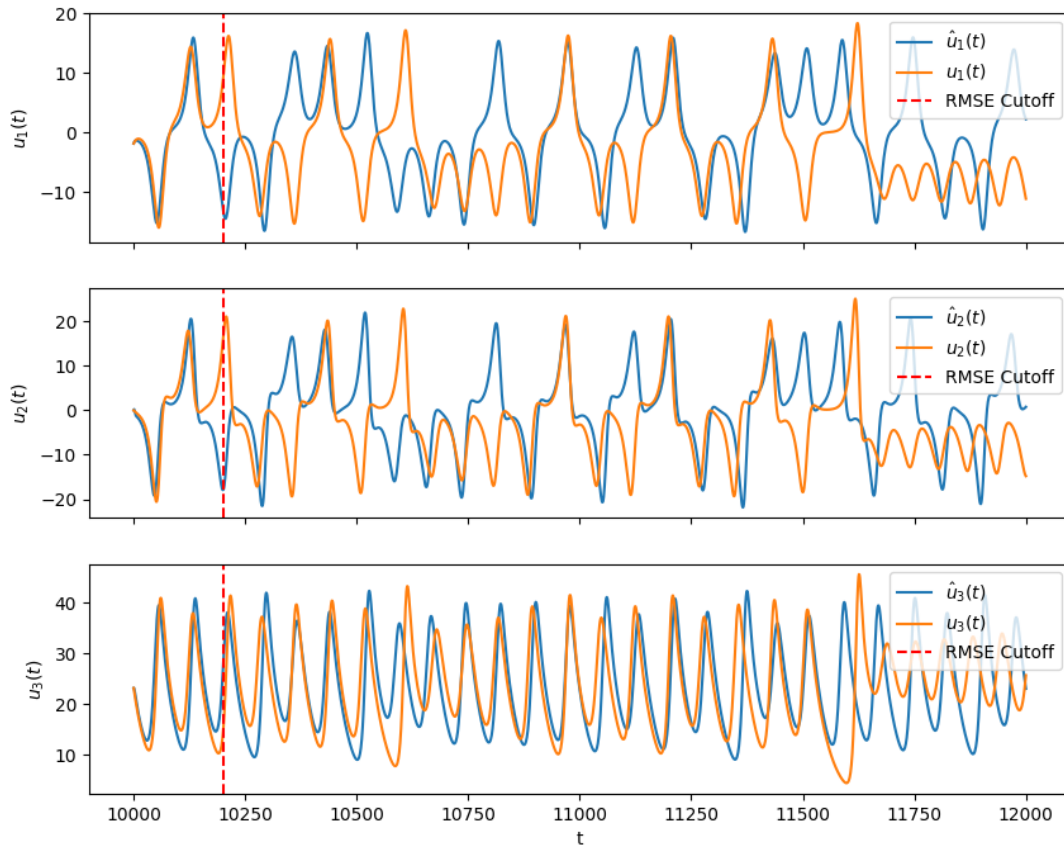
```

axes[i].legend(loc='upper right')

plt.xlabel("t")
plt.savefig("lorenz_prediction.png")
plt.show()

```

Component-wise plot of true vs predicted data



B.9 A 3D plot of true data vs predicted data

```

[10]: ax3d = plt.axes(projection='3d')
ax3d.plot3D(true_data[:,0], true_data[:,1], true_data[:,2],
            label=r'$u(t)$')
ax3d.plot3D(pred_data[:,0], pred_data[:,1], pred_data[:,2],
            label=r'$\hat{u}(t)$')
ax3d.legend(loc="upper right")

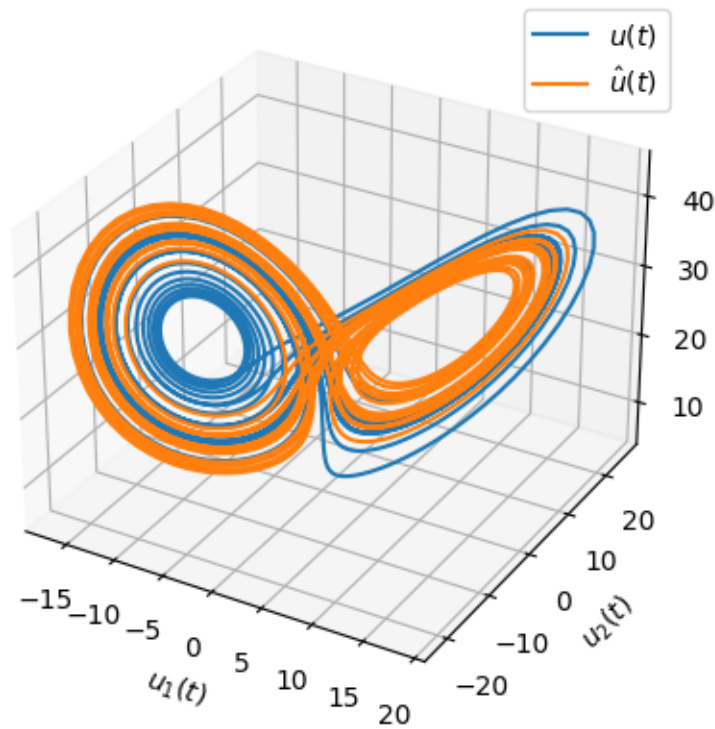
plt.title("3D Plot of true vs predicted data")
plt.savefig("lorenz_prediction_3d.png")

```

```
ax3d.set_xlabel(r'$u_1(t)$')
ax3d.set_ylabel(r'$u_2(t)$')
ax3d.set_zlabel(r'$u_3(t)$')
```

```
plt.show()
```

3D Plot of true vs predicted data



B.10 Computing the Root Mean Square Error of true data vs predicted data

```
[11]: rmse = np.sqrt(sum(((true_data[:rmse_cutoff] - pred_data[:rmse_cutoff])  
    ↪** 2) / rmse_cutoff))  
print(f"Root Mean Square Error:", rmse)
```

Root Mean Square Error: [4.57865905 7.54930093 4.90696865]

B.11 Plot of standardised residuals until cutoff

```
[12]: residuals = standardise(true_data[:rmse_cutoff] - pred_data[:
    ↪rmse_cutoff])

# Setup the prediction range space
residuals_range = np.linspace(t_train, t_train + rmse_cutoff,
    ↪rmse_cutoff)

# Plot the predicted values against true values
fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

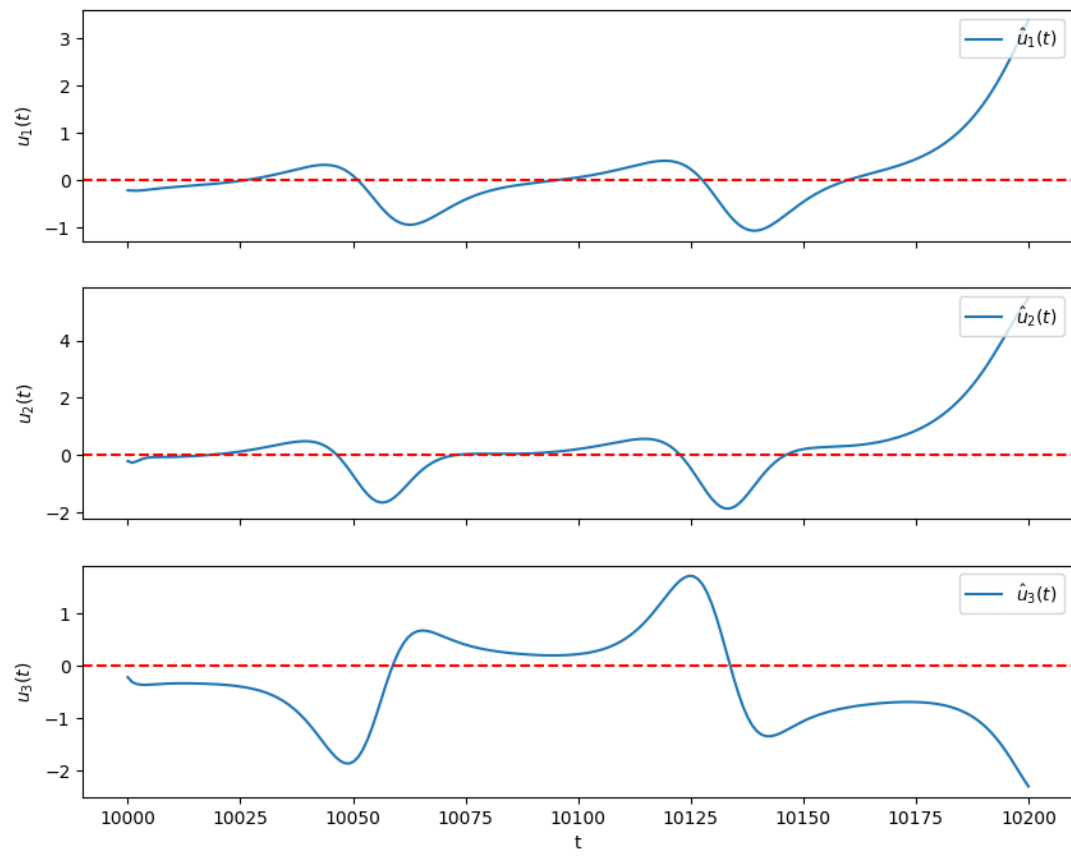
labels_pred = [r'$\hat{u}_1(t)$', r'$\hat{u}_2(t)$', r'$\hat{u}_3(t)$']

fig.suptitle("Component-wise plot of residuals")

for i in range(0,3):
    axes[i].plot(residuals_range, residuals[:,i], label=labels_pred[i])
    axes[i].set_ylabel(labels[i])
    axes[i].axhline(y=0, color='red', linestyle='--')
    axes[i].legend(loc='upper right')

plt.xlabel("t")
plt.savefig("lorenz_residuals.png")
plt.show()
```

Component-wise plot of residuals



Appendix C

NARMA10 ESN Example & Parameter Optimisation

C.1 ESN class

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import itertools
import pandas as pd
```

```
[2]: def spectral_radius(W):
    return np.max(np.abs(np.linalg.eigvals(W)))

def nmse(y_true, y_pred):
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    return np.mean((y_true - y_pred) ** 2) / np.var(y_true)

class EchoStateNetwork:
    def __init__(
        self,
        n_inputs,
        n_reservoir,
        n_outputs,
        spectral_radius_target=0.95,
        input_scale=0.6,
        leak_rate=0.3,
        ridge_alpha=1e-6,
        sparsity=0.9,
        seed=42,
        use_bias=True,
        include_inputs_in_readout=True
    ):
        self.n_reservoir = n_reservoir
```

```

self.leak_rate = leak_rate
self.ridge_alpha = ridge_alpha
self.use_bias = use_bias
self.include_inputs_in_readout = include_inputs_in_readout

rng = np.random.default_rng(seed)

self.W_in = rng.uniform(-1, 1, (n_reservoir, n_inputs)) * input_scale

W = rng.uniform(-1, 1, (n_reservoir, n_reservoir))
mask = rng.random(W.shape) < sparsity
W[mask] = 0.0
self.W_res = W * (spectral_radius_target / spectral_radius(W))

self.W_out = None

def _update(self, x, u):
    return (1 - self.leak_rate) * x + self.leak_rate * np.tanh(
        self.W_res @ x + self.W_in @ u
    )

def _feature(self, x, u):
    parts = []
    if self.use_bias:
        parts.append(np.array([1.0]))
    parts.append(x)
    if self.include_inputs_in_readout:
        parts.append(u)
    return np.concatenate(parts)

def fit(self, U, Y, washout=100):
    x = np.zeros(self.n_reservoir)
    X, Yt = [], []

    for t in range(len(U)):
        x = self._update(x, U[t])
        if t >= washout:
            X.append(self._feature(x, U[t]))
            Yt.append(Y[t])

    X = np.vstack(X)
    Yt = np.vstack(Yt)

    self.W_out = np.linalg.solve(
        X.T @ X + self.ridge_alpha * np.eye(X.shape[1]),
        X.T @ Yt
    )

```

```

    )

    def predict(self, U):
        x = np.zeros(self.n_reservoir)
        Yhat = []

        for u in U:
            x = self._update(x, u)
            Yhat.append(self._feature(x, u) @ self.W_out)

        return np.vstack(Yhat)

```

```
[ ]:
```

C.2 NARMA10 example

```

[3]: def generate_narma10(T):
    rng = np.random.default_rng(0)
    u = rng.uniform(0.0, 0.5, T)
    y = np.zeros(T)
    for t in range(10, T-1):
        y[t+1] = (
            0.3*y[t]
            + 0.05*y[t]*np.sum(y[t-9:t+1])
            + 1.5*u[t-9]*u[t]
            + 0.1
        )
    return u.reshape(-1,1), y.reshape(-1,1)

# data
U, Y = generate_narma10(4500)

split = 3000
U_train, Y_train = U[:split], Y[:split]
U_test, Y_test = U[split:], Y[split:]

# train
esn = EchoStateNetwork(
    n_inputs=1, n_reservoir=800, n_outputs=1,
    spectral_radius_target=0.9, leak_rate=0.2
)
esn.fit(U_train, Y_train, washout=200)

# predict
Y_pred = esn.predict(U_test)

```

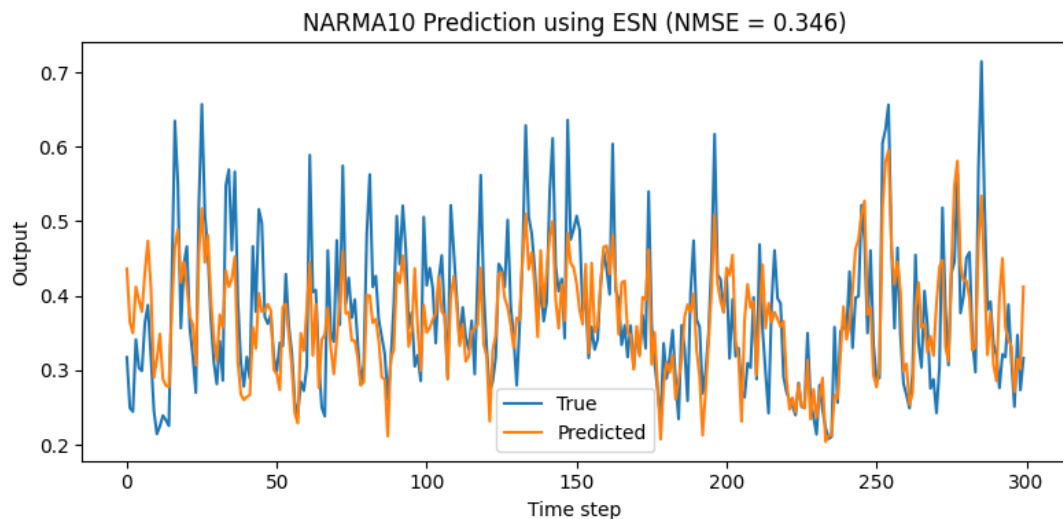
```

# NMSE
test_washout = 100
nmse_value = nmse(Y_test[test_washout:], Y_pred[test_washout:])
print("NARMA10 NMSE:", nmse_value)

# plot
start = 100 # skip early transient in the plot
end = 400
plt.figure(figsize=(8, 4))
plt.plot(Y_test[start:end], label="True")
plt.plot(Y_pred[start:end], label="Predicted")
plt.title(f"NARMA10 Prediction using ESN (NMSE = {nmse_value:.3f})")
plt.xlabel("Time step")
plt.ylabel("Output")
plt.legend()
plt.tight_layout()
plt.show()

```

NARMA10 NMSE: 0.3458505501973579



C.3 RC Implementation

```

[4]: def spectral_radius(A):
    eigvals = np.linalg.eigvals(A)
    return np.max(np.abs(eigvals))

def make_sparse_reservoir(N=300, p=0.04, rho=0.95, seed=0):
    rng = np.random.default_rng(seed)
    A = np.zeros((N, N), dtype=float)

```

```

mask = rng.random((N, N)) < p
A[mask] = rng.uniform(-1.0, 1.0, size=np.count_nonzero(mask))
# rescale to desired spectral radius
sr = spectral_radius(A)
if sr > 0:
    A *= (rho / sr)
return A

def make_Win(N, D=1, seed=1):
    rng = np.random.default_rng(seed)
    Win = np.zeros((N, D))
    # each row has one non-zero element (D=1 -> exactly one col)
    rows = np.arange(N)
    cols = rng.integers(0, D, size=N)
    Win[rows, cols] = rng.uniform(-1.0, 1.0, size=N)
    return Win

def q_feature(r):
    # q(r) in R^{2N}: [r; r^2]
    return np.concatenate([r, r**2], axis=0)

def rk4_step(f, t, y, dt):
    k1 = f(t, y)
    k2 = f(t + dt/2, y + dt*k1/2)
    k3 = f(t + dt/2, y + dt*k2/2)
    k4 = f(t + dt, y + dt*k3)
    return y + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)

[5]: def run_rc_three_stage(
    rho=0.95, N=300, D=1, p=0.04, gamma=10.0, sigma=0.5, beta=1e-4,
    dt=0.01, t_listen=2.0, t_train=4.0, t_pred=4.0,
    seed_res=0, seed_in=1, omega=2*np.pi*1.0
):
    # Build matrices
    M = make_sparse_reservoir(N=N, p=p, rho=rho, seed=seed_res)
    Win = make_Win(N=N, D=D, seed=seed_in)

    def u_of_t(t):
        return np.array([np.sin(omega * t)]) # (D,)

    # Time grid
    T_total = t_train + t_pred
    ts = np.arange(0.0, T_total + dt, dt)
    u_series = np.array([u_of_t(t) for t in ts]) # (T, D)

    # Dynamics: listening/training uses true u(t)
    def f_listening(t, r, u_t):

```

```

        return gamma * (-r + np.tanh(M @ r + sigma * (Win @ u_t)))

# --- A) Listening stage ---
r = np.zeros(N)
for k, t in enumerate(ts):
    if t > t_listen:
        break
    u_t = u_series[k].reshape(D,)
    r = rk4_step(lambda tt, rr: f_listening(tt, rr, u_t), t, r, dt)

# --- B) Training stage ---
X_cols, Y_cols = [], []
for k, t in enumerate(ts):
    if t <= t_listen:
        continue
    if t > t_train:
        break
    u_t = u_series[k].reshape(D,)
    r = rk4_step(lambda tt, rr: f_listening(tt, rr, u_t), t, r, dt)
    X_cols.append(q_feature(r))
    Y_cols.append(u_t.copy())

X = np.stack(X_cols, axis=1) # (2N, T)
Y = np.stack(Y_cols, axis=1) # (D, T)

I = np.eye(X.shape[0])
Wout = (Y @ X.T) @ np.linalg.inv(X @ X.T + beta * I) # (D, 2N)

# --- C) Predicting stage (closed loop) ---
def f_predicting(t, rhat):
    uhat = (Wout @ q_feature(rhat)).reshape(D,)
    return gamma * (-rhat + np.tanh(M @ rhat + sigma * (Win @
    uhat)))

rhat = r.copy() # initialize with r(t_train)
pred_ts = np.arange(t_train, t_train + t_pred + dt, dt)

uhat_series = []
for t in pred_ts:
    uhat = (Wout @ q_feature(rhat)).reshape(D,)
    uhat_series.append(uhat.copy())
    rhat = rk4_step(lambda tt, rr: f_predicting(tt, rr), t, rhat,
    dt)

uhat_series = np.array(uhat_series) # (T_pred, D)

# True u(t) on prediction window

```

```

mask_pred = (ts >= t_train) & (ts <= t_train + t_pred)
u_true_pred = u_series[mask_pred][:len(pred_ts), 0]
u_hat_pred = uhat_series[:, 0]

# Simple "instability" indicator: amplitude blow-up / drift
amp_ratio = (np.std(u_hat_pred) / (np.std(u_true_pred) + 1e-12))

return pred_ts, u_true_pred, u_hat_pred, amp_ratio

```

```
[ ]:
```

C.4 Closed-Loop Prediction and Parameter Optimisation

```
[ ]: Appendix A.4 Closed-loop prediction and parameter optimisation
```

```
[7]: # Layer 1: Utility functions
```

```

def spectral_radius(A):
    eigenvalues = np.linalg.eigvals(A)
    return np.max(np.abs(eigenvalues))

def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

def nmse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2) / np.var(y_true)

def r2_score_manual(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - np.mean(y_true, axis=0)) ** 2)
    return 1 - ss_res / ss_tot

# Layer 2: Generate signal
# Slightly richer than a single sine wave
T = 2000
t = np.linspace(0, 80, T)
signal = np.sin(t) + 0.5 * np.sin(2.3 * t) + 0.2 * np.cos(0.7 * t)

signal = signal.reshape(-1, 1)

# We use the same sequence for one-step-ahead training:
# input at time t -> target at time t+1
U = signal[:-1]
Y = signal[1:]

```

```

# Layer 3: Important parameters

K = 1
L = 1
N = 200

sigma_in = 0.4
rho_target = 0.7
sparsity = 0.1
beta_ridge = 1e-4

t_listen = 100
t_train_end = 1200
predict_steps = 400

# Layer 4: Initialise reservoir

def initialise_reservoir(K, N, sigma_in=0.4, rho_target=0.7, sparsity=0.
    ↪1, seed=42):
    rng = np.random.default_rng(seed)

    W_in = rng.uniform(-1, 1, size=(N, K))
    W_in = sigma_in * W_in

    W_res = rng.uniform(-1, 1, size=(N, N))
    mask = rng.random((N, N)) < sparsity
    W_res = W_res * mask

    rho = spectral_radius(W_res)
    if rho > 0:
        W_res = W_res * (rho_target / rho)

    return W_in, W_res

W_in, W_res = initialise_reservoir(
    K=K,
    N=N,
    sigma_in=sigma_in,
    rho_target=rho_target,
    sparsity=sparsity,
    seed=42
)

# Layer 5: State update equation

```



```

def update_state(r_t, u_t, W_in, W_res):
    return np.tanh(W_res @ r_t + W_in @ u_t)

# Layer 6: Listening stage
# Drive reservoir with true input for washout

def listening_stage(U, W_in, W_res, t_listen):
    r = np.zeros((W_res.shape[0], 1))

    for t in range(t_listen):
        u_t = U[t].reshape(-1, 1)
        r = update_state(r, u_t, W_in, W_res)

    return r

r_after_listen = listening_stage(U, W_in, W_res, t_listen)

# Layer 7: Training stage
# Collect states and solve ridge regression
# x_t = [1, u(t), r(t)]

def training_stage(U, Y, W_in, W_res, t_start, t_end, beta_ridge, r_start):
    r = r_start.copy()
    K = U.shape[1]
    L = Y.shape[1]

    X_list = []
    Y_list = []

    for t in range(t_start, t_end):
        u_t = U[t].reshape(K, 1)
        r = update_state(r, u_t, W_in, W_res)

        x_t = np.vstack([
            np.ones((1, 1)),
            u_t,
            r
        ])

        X_list.append(x_t)
        Y_list.append(Y[t].reshape(L, 1))

    X = np.hstack(X_list)
    Y_target = np.hstack(Y_list)

```

```

I = np.eye(X.shape[0])
W_out = Y_target @ X.T @ np.linalg.inv(X @ X.T + beta_ridge * I)

return W_out, r

W_out, r_after_training = training_stage(
    U=U,
    Y=Y,
    W_in=W_in,
    W_res=W_res,
    t_start=t_listen,
    t_end=t_train_end,
    beta_ridge=beta_ridge,
    r_start=r_after_listen
)

# Layer 8: Predicting stage
# True closed-loop recursive prediction
# Only the first input is taken from the true signal.
# After that, the model feeds back its own prediction.

def closed_loop_prediction(W_in, W_res, W_out, r_start, u_init, steps):
    r = r_start.copy()
    u_t = u_init.reshape(-1, 1)

    Y_pred = []

    for _ in range(steps):
        r = update_state(r, u_t, W_in, W_res)

        x_t = np.vstack([
            np.ones((1, 1)),
            u_t,
            r
        ])

        y_hat = W_out @ x_t
        Y_pred.append(y_hat.flatten())

        # feedback prediction as next input
        u_t = y_hat.copy()

    return np.array(Y_pred)

# initial input for recursive prediction

```

```

u_init = U[t_train_end].reshape(-1, 1)

Y_pred_closed = closed_loop_prediction(
    W_in=W_in,
    W_res=W_res,
    W_out=W_out,
    r_start=r_after_training,
    u_init=u_init,
    steps=predict_steps
)

Y_true_closed = Y[t_train_end:t_train_end + predict_steps]

# Layer 9: Evaluation
print("Main parameters")
print("N =", N)
print("sigma_in =", sigma_in)
print("rho_target =", rho_target)
print("sparsity =", sparsity)
print("beta_ridge =", beta_ridge)
print("t_listen =", t_listen)
print("t_train_end =", t_train_end)
print("predict_steps =", predict_steps)

print("\nClosed-loop prediction performance")
print("RMSE =", rmse(Y_true_closed, Y_pred_closed))
print("NMSE =", nmse(Y_true_closed, Y_pred_closed))
print("R^2 =", r2_score_manual(Y_true_closed, Y_pred_closed))

# Layer 10: Residual analysis

residuals_closed = Y_true_closed.flatten() - Y_pred_closed.flatten()

# Layer 11: Visualisation

plt.figure(figsize=(10, 4))
plt.plot(Y_true_closed, label="True")
plt.plot(Y_pred_closed, label="Closed-loop predicted", linestyle="--")
plt.title("Closed-loop recursive prediction")
plt.xlabel("Time step")
plt.ylabel("Output")
plt.legend()
plt.tight_layout()
plt.show()

```

```

plt.figure(figsize=(10, 4))
plt.plot(residuals_closed)
plt.axhline(y=0, linestyle="--")
plt.title("Residuals for closed-loop prediction")
plt.xlabel("Time step")
plt.ylabel("Residual")
plt.tight_layout()
plt.show()

```

Main parameters

```

N = 200
sigma_in = 0.4
rho_target = 0.7
sparsity = 0.1
beta_ridge = 0.0001
t_listen = 100
t_train_end = 1200
predict_steps = 400

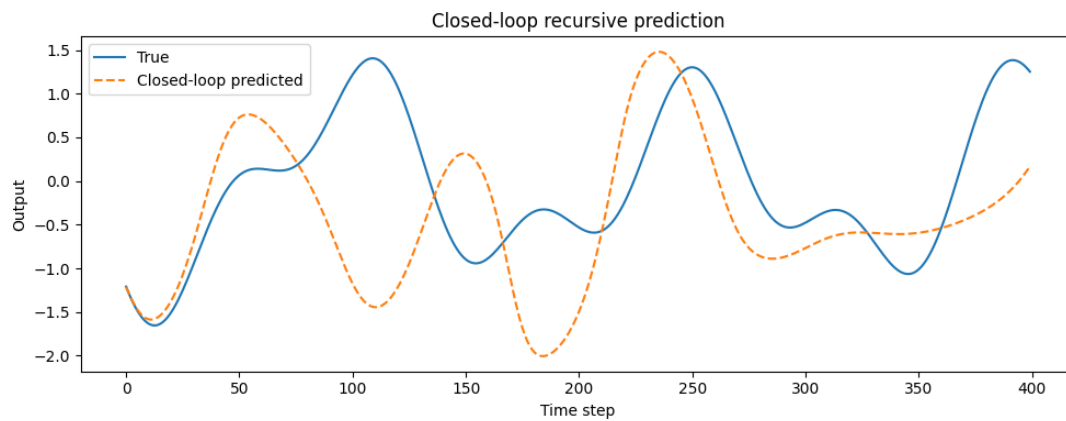
```

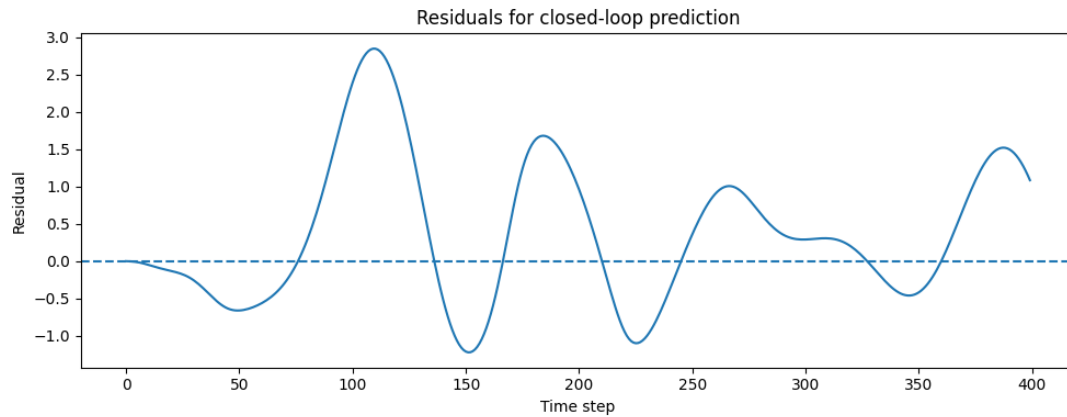
Closed-loop prediction performance

```

RMSE = 1.020704237678332
NMSE = 1.562852177154518
R^2 = -0.5628521771545181

```





[8]: *#1. Wrap the whole experiment into one function*

```
def run_rc_experiment(
    N=200,
    sigma_in=0.4,
    rho_target=0.7,
    sparsity=0.1,
    beta_ridge=1e-4,
    t_listen=100,
    t_train_end=1200,
    predict_steps=400,
    seed=42
):
    # Initialise reservoir
    W_in, W_res = initialise_reservoir(
        K=K,
        N=N,
        sigma_in=sigma_in,
        rho_target=rho_target,
        sparsity=sparsity,
        seed=seed
    )

    # Listening stage
    r_after_listen = listening_stage(U, W_in, W_res, t_listen)

    # Training stage
    W_out, r_after_training = training_stage(
        U=U,
        Y=Y,
        W_in=W_in,
        W_res=W_res,
        t_start=t_listen,
```

```

        t_end=t_train_end,
        beta_ridge=beta_ridge,
        r_start=r_after_listen
    )

    # Closed-loop prediction
    u_init = U[t_train_end].reshape(-1, 1)

    Y_pred_closed = closed_loop_prediction(
        W_in=W_in,
        W_res=W_res,
        W_out=W_out,
        r_start=r_after_training,
        u_init=u_init,
        steps=predict_steps
    )

    Y_true_closed = Y[t_train_end:t_train_end + predict_steps]

    # Metrics
    result = {
        "N": N,
        "sigma_in": sigma_in,
        "rho_target": rho_target,
        "sparsity": sparsity,
        "beta_ridge": beta_ridge,
        "t_listen": t_listen,
        "t_train_end": t_train_end,
        "predict_steps": predict_steps,
        "RMSE": rmse(Y_true_closed, Y_pred_closed),
        "NMSE": nmse(Y_true_closed, Y_pred_closed),
        "R2": r2_score_manual(Y_true_closed, Y_pred_closed)
    }

    return result, Y_true_closed, Y_pred_closed

# 2. Define parameter search space
# Keep it moderate first, otherwise it may take long

N_list = [100, 200, 300]
sigma_list = [0.2, 0.3, 0.4]
rho_list = [0.5, 0.6, 0.7, 0.8]
sparsity_list = [0.05, 0.1]
beta_list = [1e-4, 1e-3, 1e-2]
t_listen_list = [50, 100, 150]

# keep these fixed for now

```

```

t_train_end_fixed = 1200
predict_steps_fixed = 400

# 3. Grid search

all_results = []

param_grid = list(itertools.product(
    N_list,
    sigma_list,
    rho_list,
    sparsity_list,
    beta_list,
    t_listen_list
))

print("Total combinations:", len(param_grid))

for i, (N_val, sigma_val, rho_val, sparsity_val, beta_val,
      t_listen_val) in enumerate(param_grid, 1):
    result, _, _ = run_rc_experiment(
        N=N_val,
        sigma_in=sigma_val,
        rho_target=rho_val,
        sparsity=sparsity_val,
        beta_ridge=beta_val,
        t_listen=t_listen_val,
        t_train_end=t_train_end_fixed,
        predict_steps=predict_steps_fixed,
        seed=42
    )

    all_results.append(result)

    if i % 10 == 0 or i == len(param_grid):
        print(f"Finished {i}/{len(param_grid)}")

results_df = pd.DataFrame(all_results)

# 4. Sort results

# best by RMSE
best_by_rmse = results_df.sort_values(by="RMSE", ascending=True).
    reset_index(drop=True)

```

```

# best by NMSE
best_by_nmse = results_df.sort_values(by="NMSE", ascending=True).
    ↪reset_index(drop=True)

# best by R2
best_by_r2 = results_df.sort_values(by="R2", ascending=False).
    ↪reset_index(drop=True)

print("\nBest 10 by RMSE")
print(best_by_rmse.head(10))

print("\nBest 10 by NMSE")
print(best_by_nmse.head(10))

print("\nBest 10 by R2")
print(best_by_r2.head(10))

# 5. Choose one best parameter set
#     Here we use RMSE as the main criterion

best_params = best_by_rmse.iloc[0].to_dict()

print("\nBest parameter set based on RMSE:")
for k, v in best_params.items():
    print(f"{k}: {v}")

# 6. Re-run best model and plot

best_result, Y_true_best, Y_pred_best = run_rc_experiment(
    N=int(best_params["N"]),
    sigma_in=float(best_params["sigma_in"]),
    rho_target=float(best_params["rho_target"]),
    sparsity=float(best_params["sparsity"]),
    beta_ridge=float(best_params["beta_ridge"]),
    t_listen=int(best_params["t_listen"]),
    t_train_end=int(best_params["t_train_end"]),
    predict_steps=int(best_params["predict_steps"]),
    seed=42
)

print("\nBest model performance")
print("RMSE =", best_result["RMSE"])
print("NMSE =", best_result["NMSE"])
print("R2   =", best_result["R2"])

```


Total combinations: 648

Finished 10/648
Finished 20/648
Finished 30/648
Finished 40/648
Finished 50/648
Finished 60/648
Finished 70/648
Finished 80/648
Finished 90/648
Finished 100/648
Finished 110/648
Finished 120/648
Finished 130/648
Finished 140/648
Finished 150/648
Finished 160/648
Finished 170/648
Finished 180/648
Finished 190/648
Finished 200/648
Finished 210/648
Finished 220/648
Finished 230/648
Finished 240/648
Finished 250/648
Finished 260/648
Finished 270/648
Finished 280/648
Finished 290/648
Finished 300/648
Finished 310/648
Finished 320/648
Finished 330/648
Finished 340/648
Finished 350/648
Finished 360/648
Finished 370/648
Finished 380/648
Finished 390/648
Finished 400/648
Finished 410/648
Finished 420/648
Finished 430/648
Finished 440/648
Finished 450/648
Finished 460/648
Finished 470/648

Finished 480/648
 Finished 490/648
 Finished 500/648
 Finished 510/648
 Finished 520/648
 Finished 530/648
 Finished 540/648
 Finished 550/648
 Finished 560/648
 Finished 570/648
 Finished 580/648
 Finished 590/648
 Finished 600/648
 Finished 610/648
 Finished 620/648
 Finished 630/648
 Finished 640/648
 Finished 648/648

Best 10 by RMSE

	N	sigma_in	rho_target	sparsity	beta_ridge	t_listen	↵
↵t_train_end \							
0	300	0.2	0.8	0.10	0.0010	50	1200
1	300	0.2	0.7	0.10	0.0001	50	1200
2	200	0.2	0.8	0.05	0.0001	150	1200
3	300	0.2	0.8	0.10	0.0001	150	1200
4	100	0.4	0.8	0.10	0.0010	50	1200
5	300	0.3	0.8	0.10	0.0010	50	1200
6	200	0.2	0.8	0.05	0.0001	100	1200
7	100	0.2	0.5	0.05	0.0100	150	1200
8	100	0.2	0.8	0.10	0.0001	50	1200
9	100	0.4	0.8	0.10	0.0010	100	1200

	predict_steps	RMSE	NMSE	R2
0	400	0.478650	0.343680	0.656320
1	400	0.721410	0.780698	0.219302
2	400	0.737678	0.816304	0.183696
3	400	0.745190	0.833013	0.166987
4	400	0.745734	0.834230	0.165770
5	400	0.762639	0.872482	0.127518
6	400	0.767907	0.884576	0.115424
7	400	0.778872	0.910020	0.089980
8	400	0.787649	0.930645	0.069355
9	400	0.794345	0.946533	0.053467

Best 10 by NMSE

N	sigma_in	rho_target	sparsity	beta_ridge	t_listen	↵
↵t_train_end \						

0	300	0.2	0.8	0.10	0.0010	50	1200
1	300	0.2	0.7	0.10	0.0001	50	1200
2	200	0.2	0.8	0.05	0.0001	150	1200
3	300	0.2	0.8	0.10	0.0001	150	1200
4	100	0.4	0.8	0.10	0.0010	50	1200
5	300	0.3	0.8	0.10	0.0010	50	1200
6	200	0.2	0.8	0.05	0.0001	100	1200
7	100	0.2	0.5	0.05	0.0100	150	1200
8	100	0.2	0.8	0.10	0.0001	50	1200
9	100	0.4	0.8	0.10	0.0010	100	1200

	predict_steps	RMSE	NMSE	R2
0	400	0.478650	0.343680	0.656320
1	400	0.721410	0.780698	0.219302
2	400	0.737678	0.816304	0.183696
3	400	0.745190	0.833013	0.166987
4	400	0.745734	0.834230	0.165770
5	400	0.762639	0.872482	0.127518
6	400	0.767907	0.884576	0.115424
7	400	0.778872	0.910020	0.089980
8	400	0.787649	0.930645	0.069355
9	400	0.794345	0.946533	0.053467

Best 10 by R2

	N	sigma_in	rho_target	sparsity	beta_ridge	t_listen	
	t_train_end	\					
0	300	0.2	0.8	0.10	0.0010	50	1200
1	300	0.2	0.7	0.10	0.0001	50	1200
2	200	0.2	0.8	0.05	0.0001	150	1200
3	300	0.2	0.8	0.10	0.0001	150	1200
4	100	0.4	0.8	0.10	0.0010	50	1200
5	300	0.3	0.8	0.10	0.0010	50	1200
6	200	0.2	0.8	0.05	0.0001	100	1200
7	100	0.2	0.5	0.05	0.0100	150	1200
8	100	0.2	0.8	0.10	0.0001	50	1200
9	100	0.4	0.8	0.10	0.0010	100	1200

	predict_steps	RMSE	NMSE	R2
0	400	0.478650	0.343680	0.656320
1	400	0.721410	0.780698	0.219302
2	400	0.737678	0.816304	0.183696
3	400	0.745190	0.833013	0.166987
4	400	0.745734	0.834230	0.165770
5	400	0.762639	0.872482	0.127518
6	400	0.767907	0.884576	0.115424
7	400	0.778872	0.910020	0.089980
8	400	0.787649	0.930645	0.069355
9	400	0.794345	0.946533	0.053467

Best parameter set based on RMSE:

N: 300.0

sigma_in: 0.2

rho_target: 0.8

sparsity: 0.1

beta_ridge: 0.001

t_listen: 50.0

t_train_end: 1200.0

predict_steps: 400.0

RMSE: 0.4786503081814809

NMSE: 0.3436803897095677

R2: 0.6563196102904323

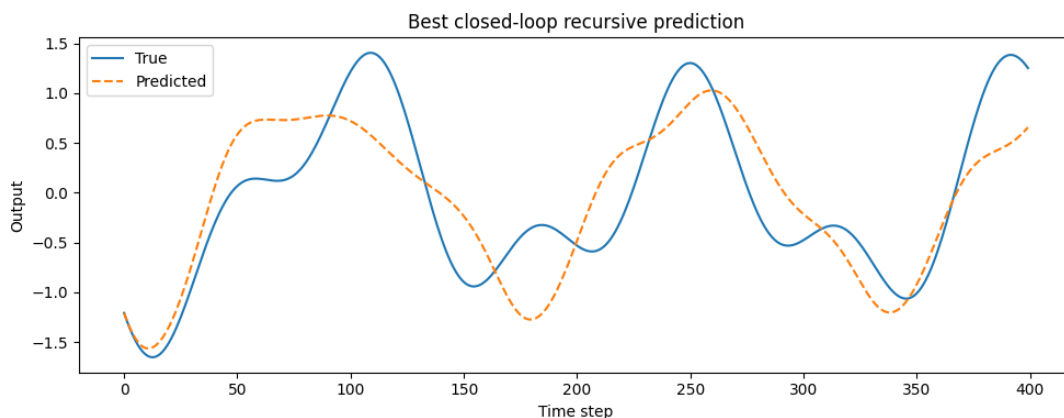
Best model performance

RMSE = 0.4786503081814809

NMSE = 0.3436803897095677

R2 = 0.6563196102904323

```
[9]: # Plot prediction
plt.figure(figsize=(10, 4))
plt.plot(Y_true_best, label="True")
plt.plot(Y_pred_best, label="Predicted", linestyle="--")
plt.title("Best closed-loop recursive prediction")
plt.xlabel("Time step")
plt.ylabel("Output")
plt.legend()
plt.tight_layout()
plt.show()
```



```
[ ]:
```

```
[ ]:
```